📁 `src/auth/` — Authentication Module

Task 1: Google OAuth + Standard Auth

`POST /api/auth/register`

Input:

json

```
JSON
{ "email": "string", "password": "string", "full_name": "string",
"workspace_id": "uuid" }
```

Logic: Hash password with bcrypt. Create `users` row with role `team_member` by default. Send email verification. Return tokens. **Output:** `{ access_token, refresh_token, user: { id, email, full_name, role } }`

`POST /api/auth/login`

Input: `{ "email": "string", "password": "string" }` **Logic:** Find user by email, compare bcrypt hash, check `is_active`. Issue JWT access token (15min) + refresh token (7d). Store refresh token in `refresh_tokens` table. **Output:** `{ access_token, refresh_token, user: { id, email, full_name, role, workspace_id } }`

`POST /api/auth/google`

Input: `{ "code": "string" }` — OAuth authorization code from frontend **Logic:** Exchange code with Google for ID token. Verify token. If user doesn't exist → create user with `google_id`, no password hash. If exists → update `google_id`. Issue JWT + refresh token same as login. **Output:** `{ access_token, refresh_token, user: { id, email, full_name, role } }`

`POST /api/auth/refresh`

Input: `{ "refresh_token": "string" }` **Logic:** Look up refresh token in `refresh_tokens` table. Validate it's not expired/revoked. Issue new access token. Optionally rotate refresh token. **Output:** `{ access_token, refresh_token }`

`POST /api/auth/logout`

Input: Bearer token in header **Logic:** Find the refresh token record for this session, mark as revoked (or delete). Invalidate current session. **Output:** `{ message: "Logged out" }`

`GET /api/auth/me`

Input: Bearer token in header **Logic:** Decode JWT, fetch user from DB including workspace. Return user's role, assigned projects with their access levels from `project_members`. **Output:** `{ id, email, full_name, role, workspace_id, avatar_url, default_model, assigned_projects: [...] }`

📁 `src/api-keys/` — BYOK Module

Task 2: Bring Your Own Key

`POST /api/admin/api-keys`

Input:

json

```
JSON
{ "provider": "anthropic | openai | google", "key": "string" }
```

Logic:

1. Guard: Admin only.
2. Immediately validate the key against the provider's API (e.g. hit Anthropic's `/models` endpoint with the key).
3. If validation fails → save with `key_status: "invalid"`, return error message.
4. If valid → encrypt key using AES-256 (store in `encrypted_key`), set `key_status: "connected"`, save `last_validated_at`.
5. Write audit log entry.
6. **Never return the raw key in any response.**

Output: `{ id, provider, key_status: "connected" | "invalid", last_validated_at }`

`GET /api/admin/api-keys`

Input: Bearer token (Admin only) **Logic:** Fetch all `api_keys` for the workspace. Strip `encrypted_key` from response entirely. **Output:**

json

```
JSON
[{ "id": "uuid", "provider": "anthropic", "key_status":
"connected", "last_validated_at": "...", "added_by": "uuid" }]
```

`PATCH /api/admin/api-keys/:id`

Input: `{ "key": "string" }` — new key value **Logic:**

1. Admin guard.
2. Validate new key against provider immediately.
3. Re-encrypt and update `encrypted_key`. Update `key_status` and `last_validated_at`.
4. Does NOT disrupt active sessions — existing threads will pick up new key on next message.

Output: `{ id, provider, key_status, last_validated_at }`

`DELETE /api/admin/api-keys/:id`

Input: Param `:id` **Logic:** Admin guard. Delete the key row. Set `key_status` to `"missing"` (or just delete and treat absence as missing at model router level). **Output:** `{ message: "Key removed" }`

`POST /api/admin/api-keys/:id/validate`

Input: Param `:id` **Logic:** Fetch and decrypt the key. Hit provider's health endpoint. Update `key_status` and `last_validated_at` accordingly. **Output:** `{ key_status: "connected" | "invalid" }`

> **Important note for model router:** When a thread message is sent and the provider key is `missing` or `invalid`, the router must return a **422** with `{`

`error: "KEY_MISSING", message: "This model is not available – admin needs to connect the [Provider] API key." }`. The frontend maps this to a visible inline error in the chat.

📁 `src/users/` — User & Role Management Module

Part of Task 3: Project Owner role management

`GET /api/admin/users`

Input: Optional query params: `?search=`, `?role=`, `?is_active=` **Logic:** Admin only. List all users in workspace with filters. **Output:** `[{ id, full_name, email, role, is_active, created_at }]`

`POST /api/admin/users`

Input: `{ "email": "string", "full_name": "string", "role": "team_member" }` **Logic:** Admin only. Create user, send invite email with password-set link. **Output:** `{ id, email, full_name, role }`

`PATCH /api/admin/users/:id/promote`

Input: No body needed (or optionally `{ "role": "project_owner" }`) **Logic:**

1. Admin only.
2. Change `users.role` from `team_member` → `project_owner`.
3. Write audit log.

Output: `{ id, email, role: "project_owner" }`

`PATCH /api/admin/users/:id/demote`

Input: No body **Logic:**

1. Admin only.
2. Find all projects where `projects.owner_id = :id`.

3. For each such project → set `projects.owner_id = requesting_admin_id` (admin takes ownership gracefully). Admin sets who to promote to the owner of that specific project
4. Then change `users.role` → `team_member`.
5. Write audit log entry noting ownership transfers.

Output: `{ id, email, role: "team_member", transferred_projects: ["proj_id_1", "proj_id_2"] }`

`PATCH /api/admin/users/:id/deactivate`

Input: No body **Logic:** Set `is_active = false`. Does not delete data. User can no longer log in. **Output:** `{ id, is_active: false }`

`Give a option to activate the user account once deactivated.`

📁 `src/projects/` — Project Management Module

Task 3: Project endpoints

`GET /api/projects`

Input: Query params — all optional:

```
None
?search=          // searches name + brief (live search)
?status=          // active | paused | completed | archived
?type=            // client | internal_bd | internal_build
?owner_id=        // filter by project owner
?date_from=       // created_at range start
?date_to=         // created_at range end
?has_linked_sources=true  // has any of clickup/slack/fathom
links populated
?group_by=        // none | owner | last_active | monthly_cost
?sort_by=         // last_active (default) | name_asc |
cost_high_low | thread_count
```

Logic:

1. If Admin → return all projects in workspace.
2. If Project Owner or Team Member → return only projects they are a member of via `project_members`.
3. Apply all filter/sort/group query params in a single DB query. No separate filter endpoints.
4. Include `thread_count` (count of threads), `monthly_cost` (sum of thread costs this month), `member_count`.

Output:

json

```
JSON
[{
  "id": "uuid", "name": "string", "type": "client", "status": "active",
  "owner": { "id": "uuid", "full_name": "string", "avatar_url": "string" },
  "thread_count": 12, "monthly_cost": "24.50", "last_active_at": "...",
  "linked_sources": { "clickup": true, "slack": false, "fathom": false }
}]
```

`POST /api/projects`

Input:

json

```
JSON
{
  "name": "string",
  "type": "client | internal_bd | internal_build",
  "status": "active | paused | completed | archived",
  "brief": "string",
  "system_instructions": "string",
  "default_model": "string",
  "members": [{ "user_id": "uuid", "access_level": "read_only | edit" }],
  "clickup_link": "string?",
  "slack_channel_link": "string?",
  "fathom_link": "string?",
```

```
  "vault_drive_link": "string?"
}
```

Logic:

1. Guard: Admin or Project Owner only.
2. Create `projects` row with `owner_id = requesting_user.id`.
3. Insert each member in `project_members` with `assigned_by = requesting_user.id`.
4. Creator is auto-added as owner (already via `owner_id`).
5. Write audit log.

Output: Full project object including members list.

`GET /api/projects/:id`

Input: Param `:id` **Logic:** Check user has access to this project (is member OR admin). Return full project detail with members, linked sources, and settings. **Output:** Full project object + `{ members: [...], custom_properties: [...] }`

`PATCH /api/projects/:id`

Input: Any subset of:

json

```JSON
{
  "name": "string", "brief": "string", "system_instructions": "string",
  "status": "string", "default_model": "string",
  "clickup_link": "string", "slack_channel_link": "string",
  "fathom_link": "string", "vault_drive_link": "string"
}
```

Logic: Admin or Project Owner of this specific project only. Patch only provided fields. Write audit log. **Output:** Updated project object.

`POST /api/projects/:id/archive - frontend missing`

Input: No body **Logic:** Admin only. Set `status = "archived"`. Data fully retained. Project disappears from active views (filtered out by default in `GET /api/projects` unless `?status=archived` passed). **Output:** `{ id, status: "archived" }`

`DELETE /api/projects/:id`

Input: No body **Logic:** Admin only. Hard delete or soft delete depending on your policy. Dangerous — require confirmation token in header or body. Write audit log. **Output:** `{ message: "Project deleted" }`

`POST /api/projects/:id/members`

Input: `{ "user_id": "uuid", "access_level": "read_only | edit" }` **Logic:** Admin or Project Owner. Verify user is in same workspace. Insert into `project_members`. If already a member → update `access_level`. **Output:** `{ project_id, user_id, access_level, assigned_at }`

`PATCH /api/projects/:id/members/:userId`

Input: `{ "access_level": "read_only | edit" }` **Logic:** Admin or Project Owner. Update the `access_level` in `project_members`. **Output:** Updated member record.

`DELETE /api/projects/:id/members/:userId`

Input: Params only **Logic:** Admin or Project Owner. Cannot remove the `owner_id` of the project this way — must transfer ownership first. Remove row from `project_members`. **Output:** `{ message: "Member removed" }`

`PATCH /api/projects/:id/transfer-ownership`

Input: `{ "new_owner_id": "uuid" }` **Logic:** Admin only. Update `projects.owner_id`. Ensure new owner is in the workspace. Write audit log. **Output:** Updated project with new owner.

`GET /api/projects/:id/custom-properties`

Input: Param `:id` **Logic:** Any project member. Fetch all `project_custom_properties` for this project ordered by `sort_order`. **Output:** `[{ id, name, property_type, options, sort_order }]`

`POST /api/projects/:id/custom-properties`

Input:

json

```
JSON
{
  "name": "string",
  "property_type": "text | number | date | single_select | multi_select | checkbox | person",
  "options": [{ "value": "string", "label": "string", "color": "string" }],
  "sort_order": 0
}
```

Logic: Admin or Project Owner only. Insert into `project_custom_properties`. `options` only required for select types. **Output:** Created property object.

`PATCH /api/projects/:id/custom-properties/:propertyId`

Input: `{ "name": "string", "options": [...], "sort_order": 0 }` **Logic:** Admin or Project Owner. Update property definition. Existing `thread_property_values` are not deleted. **Output:** Updated property object.

`DELETE /api/projects/:id/custom-properties/:propertyId`

Input: Params only **Logic:** Admin or Project Owner. Delete property definition. Cascade delete all `thread_property_values` for this property. **Output:** `{ message: "Property deleted" }`

📁 `src/threads/` — Thread Module

Task 4: Thread management

`GET /api/projects/:projectId/threads`

Input: Query params — all optional:

```
None
?search=            // search thread title
?purpose_tag=       // multi-value:
?purpose_tag=Dev&purpose_tag=Copy
?created_by=        // user uuid
?model=             // filter by last_model_used
?status=            // active | archived
?date_from=
?date_to=
?cost_min=
?cost_max=
?group_id=          // filter by thread group
?sort_by=           // last_active (default) | title_asc |
cost_desc | created_at
?include_private=   // false by default; true only for
admin/creator
```

Logic:

1. Verify user is a member of this project.
2. Apply AND logic on all provided filters in single query.
3. **Private threads** — only return if `access_level = "team"` OR `created_by = requesting_user.id` OR user is Admin.
4. Join `thread_property_values` so custom property values come back per thread.
5. Return threads grouped by `group_id` if relevant (frontend will handle collapsible groups).

Output:

json

```
JSON
{
  "groups": [{ "id": "uuid", "name": "string", "total_cost":
"12.50", "threads": [...] }],
  "ungrouped_threads": [...]
}
```

Each thread object: `{ id, title, purpose_tag, status, access_level, last_model_used, created_by, last_active_at, total_cost, property_values: { property_id: value } }`

`POST /api/projects/:projectId/threads`

Input:

json

```
JSON
{
  "title": "string",
  "purpose_tag": "string?",
  "model": "string",
  "system_prompt": "string?",
  "access_level": "team | private",
  "group_id": "uuid?",
  "skill_ids": ["uuid"]
}
```

Logic:

1. Verify user has `edit` access to this project.
2. Insert `threads` row with `created_by = requesting_user.id`.
3. If `skill_ids` provided → insert into `thread_active_skills`.
4. Write audit log.

Output: Created thread object.

`GET /api/threads/:id`

Input: Param `:id` **Logic:** Verify access (project member + respect `access_level: private`). Return thread detail with property values. **Output:** Full thread object including `property_values` array.

`PATCH /api/threads/:id`

Input: Any subset of:

json

```JSON
{
  "title": "string",
  "purpose_tag": "string",
  "status": "active | archived",
  "access_level": "team | private",
  "group_id": "uuid | null",
  "system_prompt": "string"
}
```

Logic: Edit-access member, Admin, or Project Owner. Inline rename handled by the same endpoint (just `{ "title": "new name" }`). **Output:** Updated thread object.

`POST /api/threads/:id/property-values`

Input:

json

```JSON
{
  "values": [
    { "property_id": "uuid", "value": "any" }
  ]
}
```

Logic: Edit-access member or above. Upsert into `thread_property_values` using `@@unique([thread_id, property_id])`. Value shape depends on `property_type` of the property. **Output:** `{ thread_id, updated_values: [...] }`

📁 `src/thread-groups/` — Thread Groups Module

Kept in a **separate folder** as you requested, since groups have their own RBAC rules distinct from threads.

`POST /api/projects/:projectId/thread-groups`

Input: `{ "name": "string", "sort_order": 0 }` **Logic:**

1. **Edit access** members can create groups (not read-only, not unauthenticated).
2. Project Owner and Admin can also create.
3. Insert into `thread_groups` with `created_by = requesting_user.id`.

Output: `{ id, project_id, name, sort_order, created_by, created_at }`

`GET /api/projects/:projectId/thread-groups`

Input: Param `:projectId` **Logic:** Any project member. Return all groups for this project ordered by `sort_order`. Include `total_cost` (sum of `threads.total_cost` where `group_id = group.id`). **Output:** `[{ id, name, sort_order, total_cost, thread_count }]`

`PATCH /api/thread-groups/:id`

Input: `{ "name": "string?", "sort_order": 0? }` **Logic:**

1. **Only Admin or Project Owner** can rename or reorder groups. (Edit-access members can CREATE but not rename/delete.)
2. Update the `thread_groups` row.

Output: Updated group object.

`DELETE /api/thread-groups/:id`

Input: Param `:id` **Logic:**

1. Admin or Project Owner only.
2. Before deleting → set `group_id = null` on all threads in this group (threads become "ungrouped", not deleted).
3. Delete the group row.

Output: `{ message: "Group deleted", unassigned_thread_count: 5 }`

Folder Structure Summary

```
None
src/
```

```
├── auth/                # register, login, google oauth,
refresh, logout, me
├── api-keys/            # BYOK: add, list, update, delete,
validate
├── users/               # admin user mgmt: list, create,
promote, demote, deactivate
├── projects/            # project CRUD, members,
custom-properties, archive
├── threads/             # thread CRUD, filters, property-values
└── thread-groups/       # thread group CRUD (separate from
threads per your ask)
```

Key Patterns to Follow Across All Modules

Guard order — non-negotiable (from PRD §8.1):

```
None
1. JWT Auth guard (is the user authenticated?)
2. Role/permission guard (do they have the right role?)
3. workspace_id scoping (does this resource belong to their
workspace?)
4. Business logic
```

Never return `encrypted_key` from the `api_keys` table in any response, ever. Strip it at the service layer before returning.

Private thread rule: Always filter in SQL, not in application code — do not fetch then filter, it leaks data paths.

Audit log writes should happen at the service layer, not the controller, so they fire even if the controller has multiple code paths.

CIOS — Guards, Decorators & Auth Infrastructure

Everything you need to build once in a shared location and reuse across every module.

Folder Structure for This Layer

```
None
src/
└── common/
    ├── guards/
    │   ├── jwt-auth.guard.ts
    │   ├── roles.guard.ts
    │   ├── project-member.guard.ts
    │   ├── project-edit-access.guard.ts
    │   └── project-owner-or-admin.guard.ts
    ├── decorators/
    │   ├── current-user.decorator.ts
    │   ├── roles.decorator.ts
    │   ├── project-id.decorator.ts
    │   └── public.decorator.ts
    ├── interfaces/
    │   └── jwt-payload.interface.ts
    └── types/
        └── request-with-user.type.ts
```

---

1. JWT Payload Interface

`common/interfaces/jwt-payload.interface.ts`

This is what you encode into the JWT when issuing it at login. Every guard downstream depends on this being correct.

typescript

```
TypeScript
export interface JwtPayload {
  sub: string;          // user.id (UUID)
  email: string;
  role: 'admin' | 'team_member';
  workspace_id: string; // critical for multi-tenant scoping
  iat?: number;
  exp?: number;
}
```

> Note: `project_owner` is **not a workspace-level role** in your schema — it's a project-level relationship (`projects.owner_id`). So the JWT only carries

`admin` or `team_member`. Project ownership is resolved at the guard/service level by querying the DB.

---

2. Request With User Type

`common/types/request-with-user.type.ts`

typescript

```
TypeScript
import { Request } from 'express';
import { JwtPayload } from '../interfaces/jwt-payload.interface';

export interface RequestWithUser extends Request {
  user: JwtPayload;
}
```

This gives you a typed `req.user` everywhere — no casting.

---

3. Decorators

`@Public()` — Skip Auth on Specific Routes

`common/decorators/public.decorator.ts`

typescript

```
TypeScript
import { SetMetadata } from '@nestjs/common';
export const IS_PUBLIC_KEY = 'isPublic';
export const Public = () => SetMetadata(IS_PUBLIC_KEY, true);
```

Used on: `POST /auth/register`, `POST /auth/login`, `POST /auth/google`, `POST /auth/refresh`

`@Roles()` — Declare Required Workspace-Level Roles

`common/decorators/roles.decorator.ts`

typescript

```typescript
import { SetMetadata } from '@nestjs/common';
export type UserRole = 'admin' | 'team_member';
export const ROLES_KEY = 'roles';
export const Roles = (...roles: UserRole[]) =>
SetMetadata(ROLES_KEY, roles);
```

Usage example on a controller method:

typescript

```typescript
@Roles('admin')
@Post('api-keys')
addApiKey() {}
```

`@CurrentUser()` — Pull Authenticated User from Request

`common/decorators/current-user.decorator.ts`

typescript

```typescript
import { createParamDecorator, ExecutionContext } from
'@nestjs/common';
import { RequestWithUser } from
'../types/request-with-user.type';

export const CurrentUser = createParamDecorator(
  (_data: unknown, ctx: ExecutionContext) => {
    const request =
ctx.switchToHttp().getRequest<RequestWithUser>();
    return request.user; // the decoded JwtPayload
  },
);
```

Usage in a controller:

typescript

```typescript
@Get('me')
```

```
getMe(@CurrentUser() user: JwtPayload) {
  return this.authService.getMe(user.sub);
}
```

`@ProjectId()` — Pull Project ID from Route Params

`common/decorators/project-id.decorator.ts`

A convenience decorator so guards and services don't have to manually parse route params:

typescript

```TypeScript
import { createParamDecorator, ExecutionContext } from
'@nestjs/common';

export const ProjectId = createParamDecorator(
  (_data: unknown, ctx: ExecutionContext): string => {
    const request = ctx.switchToHttp().getRequest();
    return request.params.projectId ?? request.params.id;
  },
);
```

---

4. Guards

Guard 1: `JwtAuthGuard` — Is the User Logged In?

`common/guards/jwt-auth.guard.ts`

Applied **globally** in `AppModule`. Every endpoint is protected by default. Only routes decorated with `@Public()` bypass it.

typescript

```TypeScript
import {
  ExecutionContext,
  Injectable,
  UnauthorizedException,
```

```
} from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { AuthGuard } from '@nestjs/passport';
import { IS_PUBLIC_KEY } from '../decorators/public.decorator';

@Injectable()
export class JwtAuthGuard extends AuthGuard('jwt') {
  constructor(private reflector: Reflector) {
    super();
  }

  canActivate(context: ExecutionContext) {
    const isPublic =
this.reflector.getAllAndOverride<boolean>(IS_PUBLIC_KEY, [
      context.getHandler(),
      context.getClass(),
    ]);
    if (isPublic) return true;
    return super.canActivate(context);
  }

  handleRequest(err: any, user: any) {
    if (err || !user) throw new UnauthorizedException('Invalid or
expired token');
    return user;
  }
}
```

Register globally in `AppModule`:

typescript

```
TypeScript
providers: [{ provide: APP_GUARD, useClass: JwtAuthGuard }]
```

Guard 2: `RolesGuard` — Workspace-Level Role Check

`common/guards/roles.guard.ts`

Works alongside `@Roles()` decorator. Checks `req.user.role` against required roles.

typescript

```typescript
import { CanActivate, ExecutionContext, ForbiddenException,
Injectable } from '@nestjs/common';
import { Reflector } from '@nestjs/core';
import { ROLES_KEY, UserRole } from
'../decorators/roles.decorator';
import { RequestWithUser } from
'../types/request-with-user.type';

@Injectable()
export class RolesGuard implements CanActivate {
  constructor(private reflector: Reflector) {}

  canActivate(context: ExecutionContext): boolean {
    const requiredRoles =
this.reflector.getAllAndOverride<UserRole[]>(ROLES_KEY, [
      context.getHandler(),
      context.getClass(),
    ]);

    if (!requiredRoles || requiredRoles.length === 0) return
true;

    const { user } =
context.switchToHttp().getRequest<RequestWithUser>();

    if (!requiredRoles.includes(user.role)) {
      throw new ForbiddenException('Insufficient role');
    }
    return true;
  }
}
```

Register globally (runs after `JwtAuthGuard`):

typescript

```typescript
providers: [{ provide: APP_GUARD, useClass: RolesGuard }]
```

Guard 3: `ProjectMemberGuard` — Is the User a Member of This Project?

`common/guards/project-member.guard.ts`

This is the most important guard for multi-tenant isolation. Runs on any endpoint scoped to a project. Admins bypass the membership check (they see all projects).

typescript

```
TypeScript
import {
  CanActivate,
  ExecutionContext,
  ForbiddenException,
  Injectable,
  NotFoundException,
} from '@nestjs/common';
import { PrismaService } from 'src/prisma/prisma.service';
import { RequestWithUser } from
'../types/request-with-user.type';

@Injectable()
export class ProjectMemberGuard implements CanActivate {
  constructor(private prisma: PrismaService) {}

  async canActivate(context: ExecutionContext): Promise<boolean>
{
    const request =
context.switchToHttp().getRequest<RequestWithUser>();
    const user = request.user;

    // Resolve project ID from :projectId or :id param
    const projectId = request.params.projectId ??
request.params.id;
    if (!projectId) return true; // no project scope on this
route

    // Admins bypass membership check — they see all projects in
workspace
    if (user.role === 'admin') return true;

    // Verify project exists AND belongs to user's workspace
    const project = await this.prisma.project.findFirst({
```

```
      where: { id: projectId, workspace_id: user.workspace_id },
    });
    if (!project) throw new NotFoundException('Project not
found');

    // Check membership
    const membership = await
this.prisma.projectMember.findUnique({
      where: { project_id_user_id: { project_id: projectId,
user_id: user.sub } },
    });

    if (!membership) {
      throw new ForbiddenException('You are not a member of this
project');
    }

    // Attach membership to request so downstream guards/services
can read it
    (request as any).projectMembership = membership;

    return true;
  }
}
```

Usage — applied at controller level (not globally since not all routes are project-scoped):

typescript

```
TypeScript
@UseGuards(ProjectMemberGuard)
@Controller('projects/:projectId/threads')
export class ThreadsController {}
```

Guard 4: `ProjectEditAccessGuard` — Does the User Have Edit Access?

`common/guards/project-edit-access.guard.ts`

Used on write operations: create thread, post message, upload file, add context doc. Runs **after** `ProjectMemberGuard` (depends on `request.projectMembership` being populated).

typescript

```
TypeScript
import {
  CanActivate,
  ExecutionContext,
  ForbiddenException,
  Injectable,
} from '@nestjs/common';
import { RequestWithUser } from
'../types/request-with-user.type';

@Injectable()
export class ProjectEditAccessGuard implements CanActivate {
  canActivate(context: ExecutionContext): boolean {
    const request =
context.switchToHttp().getRequest<RequestWithUser>();
    const user = request.user;

    // Admins always have edit access
    if (user.role === 'admin') return true;

    const membership = (request as any).projectMembership;

    // ProjectMemberGuard must run first — if membership missing,
bail
    if (!membership) {
      throw new ForbiddenException('Project membership not
resolved');
    }

    if (membership.access_level !== 'edit') {
      throw new ForbiddenException(
        'You have read-only access to this project',
      );
    }

    return true;
```

```
  }
}
```

Usage on specific routes that need write access:

typescript

```TypeScript
@UseGuards(ProjectMemberGuard, ProjectEditAccessGuard)
@Post()
createThread() {}
```

Guard 5: `ProjectOwnerOrAdminGuard` — Project Settings Operations

`common/guards/project-owner-or-admin.guard.ts`

Used on: project settings, member management, custom property management, thread group rename/delete. Checks if the requesting user is either Admin (workspace-level) OR the owner of this specific project.

typescript

```TypeScript
import {
  CanActivate,
  ExecutionContext,
  ForbiddenException,
  Injectable,
  NotFoundException,
} from '@nestjs/common';
import { PrismaService } from 'src/prisma/prisma.service';
import { RequestWithUser } from
'../types/request-with-user.type';

@Injectable()
export class ProjectOwnerOrAdminGuard implements CanActivate {
  constructor(private prisma: PrismaService) {}

  async canActivate(context: ExecutionContext): Promise<boolean>
{
```

```
    const request =
context.switchToHttp().getRequest<RequestWithUser>();
    const user = request.user;

    if (user.role === 'admin') return true;

    const projectId = request.params.projectId ??
request.params.id;
    if (!projectId) throw new ForbiddenException('Project ID
missing');

    const project = await this.prisma.project.findFirst({
      where: { id: projectId, workspace_id: user.workspace_id },
    });

    if (!project) throw new NotFoundException('Project not
found');

    if (project.owner_id !== user.sub) {
      throw new ForbiddenException(
        'Only the project owner or admin can perform this
action',
      );
    }

    // Attach project to request for downstream reuse — avoids
second DB query
    (request as any).project = project;

    return true;
  }
}
```

---

5. WorkspaceScope — Always Applied in Services

Guards handle access control. Workspace isolation is a **service-layer responsibility**. Every single Prisma query must include `workspace_id`. Create a helper to enforce this:

`common/helpers/workspace-scope.helper.ts`

typescript

```typescript
import { JwtPayload } from '../interfaces/jwt-payload.interface';

/**
 * Returns a Prisma `where` fragment that always scopes to the user's workspace.
 * Use this as the base of every query — never query without it.
 *
 * Example:
 *   this.prisma.project.findMany({
 *     where: { ...workspaceScope(user), status: 'active' }
 *   })
 */
export function workspaceScope(user: JwtPayload) {
  return { workspace_id: user.workspace_id };
}
```

---

6. How They Compose — Per-Endpoint Reference

Here's how the guards layer on each endpoint category from the earlier spec:

Endpoint	Guards Applied	Notes
`POST /auth/login`	`@Public()`	No auth needed
`GET /auth/me`	`JwtAuthGuard` (global)	Just needs to be logged in
`POST /admin/api-keys`	`JwtAuthGuard` + `@Roles('admin')`	Admin only
`GET /projects`	`JwtAuthGuard`	Service filters by role + membership
`POST /projects`	`JwtAuthGuard` + service-level role check	Admin or project_owner role
`PATCH /projects/:id`	`JwtAuthGuard` + `ProjectOwnerOrAdminGuard`	Owner or admin

`POST /projects/:id/members`	`JwtAuthGuard` + `ProjectOwnerOrAdminGuard`	Owner or admin
`GET /projects/:projectId/threads`	`JwtAuthGuard` + `ProjectMemberGuard`	Any member
`POST /projects/:projectId/threads`	`JwtAuthGuard` + `ProjectMemberGuard` + `ProjectEditAccessGuard`	Edit access only
`PATCH /threads/:id`	`JwtAuthGuard` + `ProjectMemberGuard` + `ProjectEditAccessGuard`	Edit access only
`POST /projects/:id/custom-properties`	`JwtAuthGuard` + `ProjectOwnerOrAdminGuard`	Owner or admin
`POST /projects/:projectId/thread-groups`	`JwtAuthGuard` + `ProjectMemberGuard` + `ProjectEditAccessGuard`	Edit access (can create)
`PATCH /thread-groups/:id`	`JwtAuthGuard` + `ProjectOwnerOrAdminGuard`	Owner or admin only (not just edit access)
`DELETE /thread-groups/:id`	`JwtAuthGuard` + `ProjectOwnerOrAdminGuard`	Owner or admin only
`PATCH /admin/users/:id/promote`	`JwtAuthGuard` + `@Roles('admin')`	Admin only
`PATCH /admin/users/:id/demote`	`JwtAuthGuard` + `@Roles('admin')`	Admin only

---

7. Private Threads — No Guard, Handled in Service

Private threads (`access_level: 'private'`) are **not** a guard concern. They are handled inside the service query so it's enforced at the SQL level, not after data is loaded:

typescript

```typescript
// threads.service.ts — list threads
```

```
where: {
  project_id: projectId,
  workspace_id: user.workspace_id,
  OR: [
    { access_level: 'team' },
    { access_level: 'private', created_by: user.sub },
    // admin sees all — add this condition if user.role ===
'admin'
  ],
}
```

Never fetch all threads and then filter in JS. That's a data leak path.

---

8. Registration in `AppModule`

typescript

```TypeScript
providers: [
  // Global guards — order matters
  { provide: APP_GUARD, useClass: JwtAuthGuard },   // 1st — auth
  { provide: APP_GUARD, useClass: RolesGuard },      // 2nd —
workspace role

  // ProjectMemberGuard, ProjectEditAccessGuard,
ProjectOwnerOrAdminGuard
  // are NOT registered globally — applied per-controller with
@UseGuards()
  // because not every route is project-scoped
]
```

That covers the full security layer for everything in Tasks 1–4. The pattern is: **global guards for universal checks, local guards for resource-scoped checks, and service-level workspace scoping on every query.** Nothing slips through because the layers are independent — even if a guard were bypassed somehow, the `workspace_id` filter in every query stops cross-tenant data from ever appearing